

Prueba Técnica Full-Stack — App de Prescripciones

Stack obligatorio - Backend: NestJS + **Prisma** ORM + **PostgreSQL**. Autenticación con **JWT + refresh tokens**. RBAC con Guards/Decorators. - **Frontend:** **Next.js** (App Router o Pages), **React** con **TypeScript**, **TailwindCSS**. - **Infra (sugerida):** Backend en Railway/Render, DB en Railway PostgreSQL, Front en Vercel.

Objetivo: construir un MVP **simple y sólido** de un sistema de prescripciones con **3 roles: Médico, Paciente, Admin**. El médico crea prescripciones asociadas a un paciente con ítems digitados manualmente (⚠️ **no** hay CRUD de productos). El paciente ve sus prescripciones, puede **marcarlas como “consumidas”** y **descargarlas en PDF**. El admin ve **métricas** (totales y por estado).

1) Requerimientos funcionales

Roles

- **Admin**
 - Visualiza métricas: # de pacientes, # de médicos, # de prescripciones, prescripciones por **estado** y por **día**.
 - (Opcional/Plus) Crea usuarios y asigna roles.
- **Médico**
 - Crea **prescripciones** para un paciente existente (o por email del paciente).
 - Lista y ve detalle de sus propias prescripciones.
- **Paciente**
 - Lista y ve detalle de **sus** prescripciones.
 - Cambia el estado **pendiente → consumida**.
 - Descarga **PDF** de la prescripción.

Flujo mínimo

1. Autenticación por email/password.
2. Un **médico** crea una prescripción para un **paciente** (inputs libres para ítems).
3. El **paciente** ve su bandeja de prescripciones, puede marcar como **consumida** y **descargar PDF**.
4. El **admin** visualiza **métricas**.

Estados

- Prescripción: pending | consumed.

- Ítems: no requieren estado; sólo nombre, dosis, cantidad, indicaciones.
-

2) Requerimientos técnicos

Autenticación/Autorización

- **JWT** de acceso + **refresh token** (rotación recomendada). Almacenamiento seguro (HTTP-Only si usas cookies, o Bearer).
- **RBAC**: Guards con `@Roles('admin' | 'doctor' | 'patient')`.
- Rutas **protegidas** según rol (tanto en API como en Front).

Validación y seguridad

- Validación de DTOs (class-validator) y serialización (class-transformer).
- Manejo estándar de errores (filtros/exception filters) con códigos HTTP correctos.
- Seguridad básica: Helmet, CORS, rate limit básico.

Datos y DB

- PostgreSQL con **Prisma**.
- Relaciones correctas, índices en campos de búsqueda frecuentes.
- **Migraciones** con `prisma migrate` y **seed** con datos de ejemplo.

Features mínimas

- **Paginación** y **filtros** en listados (por estado, fecha).
- **Ordenamiento** (createdAt DESC por defecto; parametrizable).
- **Soft delete** (opcional) para usuarios/prescripciones.
- **PDF** (desde el backend) de una prescripción.

Testing

- **Backend**: al menos tests unitarios de servicios o e2e básicos (Jest + Supertest/Pactum).
 - **Frontend**: prueba mínima de un componente o hook crítico.
-

3) Modelado (Prisma, guía)

Puedes adaptar, pero respeta la intención.

```
// schema.prisma
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  password    String
  name        String
```

```

role      Role
createdAt DateTime @default(now())
// perfiles (opcional):
doctor    Doctor? @relation(fields: [doctorId], references: [id])
doctorId  String?
patient   Patient? @relation(fields: [patientId], references: [id])
patientId String?
prescriptionsAuthored Prescription[] @relation("AuthoredBy")
}

```

```

enum Role {
  admin
  doctor
  patient
}

```

```

model Doctor {
  id      String      @id @default(cuid())
  user    User        @relation(fields: [userId], references:
[id])
  userId  String      @unique
  specialty String?
  prescriptions Prescription[] @relation("AuthoredBy")
}

```

```

model Patient {
  id      String      @id @default(cuid())
  user    User        @relation(fields: [userId], references:
[id])
  userId  String      @unique
  birthDate DateTime?
  prescriptions Prescription[]
}

```

```

model Prescription {
  id      String      @id @default(cuid())
  code    String      @unique // para mostrar o QR en PDF
  status  PrescriptionStatus @default(pending)
  notes   String?
  createdAt DateTime   @default(now())
  consumedAt DateTime?

  // relaciones
  patient Patient      @relation(fields: [patientId],
references: [id])
  patientId String
  author  Doctor        @relation("AuthoredBy", fields:
[authorId], references: [id])
  authorId String
}

```

```

    items      PrescriptionItem[]
  }

enum PrescriptionStatus {
  pending
  consumed
}

model PrescriptionItem {
  id           String          @id @default(cuid())
  prescription Prescription @relation(fields: [prescriptionId],
references: [id])
  prescriptionId String

  name         String          // Producto digitado manualmente
  dosage       String?
  quantity     Int?           // unidades indicadas
  instructions String?
}

```

Índices sugeridos - Prescription(status, createdAt) - Prescription(patientId)
y Prescription(authorId)

4) API: contratos mínimos

Auth

- POST /auth/register (opcional si no haces panel de Admin): crea usuario **patient** o **doctor**. En la corrección se aceptan **seeds**.
- POST /auth/login → { accessToken, refreshToken }
- POST /auth/refresh → { accessToken }
- GET /auth/profile → usuario y rol.

Usuarios (mínimo para Admin — puede omitirse si usas seed)

- GET /users?role=doctor|patient&query= (paginado)
- POST /users (rol + datos básicos)

Pacientes / Doctores (si separas perfiles)

- GET /patients / GET /doctors (paginado, filtros simples)

Prescripciones

- **Médico**
 - POST /prescriptions → body:

```
{
  "patientId": "...",
  "notes": "...",
  "items": [
    { "name": "Amoxicilina 500mg", "dosage": "1 c/8h",
      "quantity": 15, "instructions": "Después de comer" }
  ]
}
```

- GET /prescriptions?mine=true&status=&from=&to=&page=&limit=&order=
- GET /prescriptions/:id

- **Paciente**

- GET /me/prescriptions?status=&page=&limit=
- PUT /prescriptions/:id/consume (marca como consumida si pertenece al paciente)
- GET /prescriptions/:id/pdf (descarga PDF si pertenece al paciente)

- **Admin**

- GET /admin/prescriptions?status=&doctorId=&patientId=&from=&to=&page=&limit=

Métricas (Admin)

- GET /admin/metrics?from=&to= →

```
{
  "totals": { "doctors": 10, "patients": 120, "prescriptions": 560 },
  "byStatus": { "pending": 120, "consumed": 440 },
  "byDay": [ { "date": "2025-10-25", "count": 20 }, ... ],
  "topDoctors": [ { "doctorId": "...", "count": 50 } ]
}
```

Reglas de acceso - Doctor: sólo ve/gestiona sus prescripciones. - **Paciente:** sólo ve/gestiona sus prescripciones. - **Admin:** acceso a todo y a métricas.

Errores - Respuestas consistentes: { message, code, details? }.
400/401/403/404/409/500 según corresponda.

5) Frontend (páginas mínimas)

Autenticación

- /login (email/password). Guarda tokens y perfil.

Médico

- /doctor/prescriptions (listado, filtros por estado/fecha, paginado)
- /doctor/prescriptions/new (formulario con ítems dinámicos **add/remove**)
- /doctor/prescriptions/[id] (detalle)

Paciente

- /patient/prescriptions (listado con acciones: **Marcar consumida**, **Descargar PDF**)
- /patient/prescriptions/[id] (detalle)

Admin

- /admin dashboard con tarjetas + gráficos simples (Recharts/Chart.js):
 - Totales (doctores, pacientes, prescripciones)
 - Por estado
 - Serie por día (últimos 30)
 - (Opcional) Top médicos por volumen

UX/UI

- **Responsive** (grid/cards); estados de **carga**, **error** y **vacío**.
 - **Toasts** para acciones (creado, consumido, error).
 - **Protección de rutas** por rol.
 - **Filtros** con persistencia en querystring.
-

6) PDF de prescripción (backend)

- Endpoint: GET /prescriptions/:id/pdf.
 - Contiene: datos del paciente, médico, fecha, código, lista de ítems (nombre, dosis, cantidad, instrucciones) y **estado**.
 - Tecnología libre: pdfkit, puppeteer/playwright + plantilla HTML, o html-pdf.
 - (Plus) **QR** con el code para abrir /patient/prescriptions/:id.
-

7) Semillas y credenciales de prueba

- Script prisma/seed.ts que cree:
 - 1 admin: admin@test.com / admin123
 - 1 médico: dr@test.com / dr123
 - 1 paciente: patient@test.com / patient123
 - 5–10 prescripciones de ejemplo (varias pending y consumed).

Se acepta no tener UI de creación de usuarios si existen **seeds**.

8) Variables de entorno (guía)

Backend (.env):

```
DATABASE_URL="postgresql://user:pass@host:5432/db?schema=public"
JWT_ACCESS_SECRET=...
JWT_REFRESH_SECRET=...
JWT_ACCESS_TTL=900s      # 15m
JWT_REFRESH_TTL=7d
APP_ORIGIN=https://frontend-url
```

Frontend (.env.local):

```
NEXT_PUBLIC_API_BASE_URL=https://backend-url
```

9) Estructura sugerida

Backend

```
src/
  main.ts
  app.module.ts
  auth/
    auth.module.ts
    auth.controller.ts
    auth.service.ts
    jwt.strategy.ts
    refresh.strategy.ts
    roles.guard.ts
  users/
    users.module.ts
    users.controller.ts
    users.service.ts
  patients/
  doctors/
  prescriptions/
    prescriptions.module.ts
    prescriptions.controller.ts
    prescriptions.service.ts
  dto/
  common/
    guards/
    interceptors/
    filters/
  prisma/
    prisma.module.ts
    prisma.service.ts
```

Frontend

```
src/  
  app/ (o pages/)  
    login/  
    doctor/prescriptions/  
      page.tsx  
    new/  
    [id]/  
  patient/prescriptions/  
    page.tsx  
    [id]/  
  admin/  
  components/  
  lib/ (fetcher, auth, guards)  
  store/ (Zustand/Redux)
```

10) Entregables

1. **Repositorio GitHub** (mono-repo o dos repos):
 - Código fuente, migraciones y **seed**.
 - **README** con:
 - setup local (Docker opcional), variables de entorno, scripts.
 - cómo correr migraciones y seed.
 - cuentas de prueba.
 2. **Despliegue funcionando**
 - URLs de Front y API en README.
 3. **Documentación**
 - Decisiones técnicas (autenticación, RBAC, generación de PDF, paginación, etc.).
 - Endpoints (OpenAPI/Swagger preferible).
 4. **Testing**
 - Comandos de test y (si aplica) reporte de coverage.
-

11) Criterios de evaluación

- **Funcionalidad (35%)**: flujos completos por rol, PDF, filtros/paginación, métricas.
- **Calidad de código (25%)**: módulos claros, DTOs/validación, manejo de errores, consistencia TS.
- **Arquitectura (20%)**: separación de capas, guards/strategies, Prisma bien usado, índices.
- **UX/UI (15%)**: responsive, estados de carga/error/vacío, toasts, DX del front.

- **Testing (5%):** mínimos que cubran lo crítico.
-

12) Plus (opcionales, para destacar)

- **Swagger** en /docs y Postman/Insomnia collection.
 - **PDF** con QR y firma/cedula del médico (texto o imagen).
 - **Auditoría** (tabla de audit logs para cambios de estado).
 - **Notificaciones por email** al crear prescripción.
 - **Búsqueda avanzada** (texto libre por nombre de ítem y notas).
 - **Tema dark/light** y preferencia persistida.
 - **SSE/WebSocket** para métricas en vivo (simple, no obligatorio).
-

13) Aceptación (checklist del revisor)

- ☐ Login funciona y devuelve perfil/rol.
 - ☐ Guardas/Decorators de roles operativos.
 - ☐ Médico crea prescripción con ítems manuales.
 - ☐ Paciente ve sólo las suyas, puede **consumir** y **descargar PDF**.
 - ☐ Admin ve métricas con filtros de fecha.
 - ☐ Listados con paginación/filtros/orden.
 - ☐ Migraciones y seed corren sin errores.
 - ☐ README suficiente para levantar en < 15 minutos.
-

14) Ejemplos de DTOs (guía)

```
// create-prescription.dto.ts
export class CreatePrescriptionDto {
  patientId: string;
  notes?: string;
  items: { name: string; dosage?: string; quantity?: number;
instructions?: string }[];
}
```

```
// update-status.dto.ts
export class ConsumePrescriptionDto {
  consumed: boolean; // true
}
```

15) Consideraciones

- No se exige un catálogo de productos: **los ítems se escriben a mano.**

- Puedes registrar pacientes/médicos via **seed**. El panel de Admin para crear usuarios es **Plus**.
- Mantén el alcance en MVP; el plus es opcional y suma puntos si el core está perfecto.

¡Demuestra tu capacidad técnica y de liderazgo creando una solución completa y profesional!